```
import javax.swing.*;
import javax.swing.Timer;
import java.awt.*;
import java.awt.event.*;
import java.util.ArrayList;
import java.util.Collections;
import java.util.List;
import java.util.Random;
import java.util.Set;
import java.util.HashSet;

public class PokerGUI extends JFrame {

    public class Card implements Comparable<Card> {
        String value; // e.g., "A","K","10"
        String suit;  // "C","D","S","H"

        public Card(String value, String suit) {
            this.value = value;
            this.suit = suit;
        }

        public String toString() {
            return value + "-" + suit;
        }

        private int getRank() {
            switch (value) {
                case "2":  return 2;
                case "3":  return 3;
                case "4":  return 4;
                case "5":  return 5;
                case "6":  return 6;
                case "7":  return 7;
                case "8":  return 8;
                case "9":  return 9;
                case "10": return 10;
                case "J":  return 11;
                case "Q":  return 12;
                case "K":  return 13;
                case "A":  return 14;
                default: return -1;
            }
        }

        @Override
        public int compareTo(Card other) {
            return Integer.compare(this.getRank(), other.getRank());
        }
    }

    public class Player {
        Card card1;
        Card card2;
        double chipstack = 1000;

        public boolean didifold = false;

        // categories (hand classification flags)
        public boolean Royal = false;
        public boolean StrFlush = false;
        public boolean FourKind = false;
```

```
    public boolean Fullhouse = false;
    public boolean Flush = false;
    public boolean Straight = false;
    public boolean threekind = false;
    public boolean twopair = false;
    public boolean pair = false;
    public boolean highcard = false;

    // per-round contribution to the pot
    public int contributed = 0;

    public String toString() {
        if (card1 == null || card2 == null) return "(no cards)";
        return card1.toString() + " & " + card2.toString();
    }
}

public class CommunityCards {
    Card card1, card2, card3, card4, card5;
    public void init(Card c1, Card c2, Card c3, Card c4, Card c5) {
        card1 = c1; card2 = c2; card3 = c3; card4 = c4; card5 = c5;
    }
    public String toString() {
        return card1 + " & " + card2 + " & " + card3 + " & " + card4 + " & " + card5;
    }
}

// ===== Deck and game state =====
private ArrayList<Card> deck = new ArrayList<>();
private List<Player> table = new ArrayList<>(); // 2 players: seat 0 and seat 1
private List<Player> active = new ArrayList<>();
private CommunityCards comm = new CommunityCards();

private Player aiPlayer;        // seat 0
private Player humanPlayer;     // seat 1

private final int aiSeat = 0;
private final int humanSeat = 1;

private int roundStage = 0; // 0 preflop, 1 flop, 2 turn, 3 river, 4 showdown-ready

// Per-player combined cards (player + community)
private ArrayList<Card> p1 = new ArrayList<>(); // seat 0 (AI)
private ArrayList<Card> p2 = new ArrayList<>(); // seat 1 (Human)

// Straight indices per player (not used heavily here)
private int idxStr1=-1, idxStr2=-1;

// Controls BB and SB amounts
private final int SMALL_BLIND = 10;
private final int BIG_BLIND = 20;

private int pot = 0;
private int currentBet = 0; // highest bet this round (amount each player must have contributed to be even)
private int currentActor = 0; // seat index of actor (0 or 1)
private boolean bettingRoundActive = false;
private boolean showdownReveal = false;

// Button (dealer) alternation
// buttonSeat indicates who is currently the button (dealer). In heads-up the button posts small blind.
// We'll alternate buttonSeat every hand.
private int buttonSeat = 1; // start with human on button (arbitrary). We'll toggle at each new deal.
```

```
    // GUI components
    private CardPanel cardPanel;
    private JButton dealBtn;
    private JLabel statusLabel;
    private JButton checkBtn, callBtn, raiseBtn, foldBtn;
    private JTextField raiseField;
    private JLabel potLabel, aiStackLabel, humanStackLabel, toCallLabel, roundLabel, buttonLabel;

    public PokerGUI() {
        setTitle("Texas Hold'em — 1v1 (auto streets, alternating blinds)");
        setSize(1000, 700);
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        setLayout(new BorderLayout());

        // Top panel for status + stacks
        JPanel top = new JPanel(new FlowLayout(FlowLayout.LEFT));
        statusLabel = new JLabel("Welcome. Click 'Deal New Hand'.");
        top.add(statusLabel);
        potLabel = new JLabel("Pot: $0");
        top.add(potLabel);
        toCallLabel = new JLabel("To Call: $0");
        top.add(toCallLabel);
        aiStackLabel = new JLabel("AI: $1000");
        top.add(aiStackLabel);
        humanStackLabel = new JLabel("You: $1000");
        top.add(humanStackLabel);
        roundLabel = new JLabel("Round: -");
        top.add(roundLabel);
        buttonLabel = new JLabel("Button: You");
        top.add(buttonLabel);

        add(top, BorderLayout.NORTH);

        // Center draw panel
        cardPanel = new CardPanel();
        add(cardPanel, BorderLayout.CENTER);

        // Betting controls (right)
        JPanel right = new JPanel();
        right.setLayout(new BoxLayout(right, BoxLayout.Y_AXIS));
        right.setPreferredSize(new Dimension(220, 0));
        right.add(Box.createVerticalStrut(10));

        checkBtn = new JButton("Check");
        callBtn = new JButton("Call");
        raiseBtn = new JButton("Bet / Raise");
        foldBtn = new JButton("Fold");
        raiseField = new JTextField("40", 6);

        checkBtn.setAlignmentX(Component.CENTER_ALIGNMENT);
        callBtn.setAlignmentX(Component.CENTER_ALIGNMENT);
        raiseField.setMaximumSize(new Dimension(160, 24));
        raiseField.setAlignmentX(Component.CENTER_ALIGNMENT);
        raiseBtn.setAlignmentX(Component.CENTER_ALIGNMENT);
        foldBtn.setAlignmentX(Component.CENTER_ALIGNMENT);

        right.add(checkBtn);
        right.add(Box.createVerticalStrut(8));
        right.add(callBtn);
        right.add(Box.createVerticalStrut(8));
        right.add(new JLabel("Bet/Raise amount:"));
```

```
        right.add(raiseField);■■■■■//right is used along the y.axis so components are stacked vertically
        right.add(Box.createVerticalStrut(8));■■■//This basically gives the gui buttons some breathing room.■■
        right.add(raiseBtn);■■■■■
        right.add(Box.createVerticalStrut(8));
        right.add(foldBtn);
        right.add(Box.createVerticalStrut(20));

        add(right, BorderLayout.EAST);

        // Bottom controls
        JPanel bottom = new JPanel(new FlowLayout());
        dealBtn = new JButton("Deal New Hand");
        bottom.add(dealBtn);
        add(bottom, BorderLayout.SOUTH);

        // Button actions
        dealBtn.addActionListener(e -> dealNewHand());

        checkBtn.addActionListener(e -> humanCheck());
        callBtn.addActionListener(e -> humanCall());
        raiseBtn.addActionListener(e -> humanRaise());
        foldBtn.addActionListener(e -> humanFold());

        // init table with 2 players
        buildPlayers(2);
        setPositions();
        updateButtons();
    }

    // ===== Core game functions mirroring Poker.java =====
    private void buildDeck() {
        deck.clear();
        String[] values = {"A","2","3","4","5","6","7","8","9","10","J","Q","K"};
        String[] suits = {"C","D","S","H"};
        for (String s : suits) {
            for (String v : values) {
                deck.add(new Card(v, s));
            }
        }
        Collections.shuffle(deck);
    }

    private void buildPlayers(int n) {
        table.clear();
        for (int i=0; i<n; i++) table.add(new Player());
        active.clear();
        active.addAll(table);
    }

    private void setPositions() {
        // Seats fixed: seat 0 = AI, seat 1 = Human
        aiPlayer = table.get(aiSeat);
        humanPlayer = table.get(humanSeat);
    }

    private void dealNewHand() {
        // Toggle button (dealer) each hand so small/blind switches
        buttonSeat = 1 - buttonSeat; // alternate
        updateButtonLabel();

        buildDeck();
        roundStage = 0;
```

```
        showdownReveal = false;

        // Clear player flags and contributions
        for (Player p : table) {
            p.didifold = false;
            p.Royal = p.StrFlush = p.FourKind = p.Fullhouse = p.Flush = p.Straight = p.threekind = p.twopair = p.pair = p.highcard = false;
            p.contributed = 0;
            p.card1 = null;
            p.card2 = null;
        }

        pot = 0;
        currentBet = 0;
        bettingRoundActive = false;

        // Burn one
        if (deck.size() > 0) deck.remove(deck.size()-1);

        // Deal 2 cards each from end (matching prior style)
        for (Player p : table) {
            p.card1 = deck.remove(deck.size()-1);
        }
        for (Player p : table) {
            p.card2 = deck.remove(deck.size()-1);
        }

        // Burn, then flop (3), burn, turn, burn, river
        deck.remove(deck.size()-1);
        Card c1 = deck.remove(deck.size()-1);
        Card c2 = deck.remove(deck.size()-1);
        Card c3 = deck.remove(deck.size()-1);
        deck.remove(deck.size()-1);
        Card c4 = deck.remove(deck.size()-1);
        deck.remove(deck.size()-1);
        Card c5 = deck.remove(deck.size()-1);

        comm.init(c1,c2,c3,c4,c5);

        // Post blinds (button posts SMALL_BLIND in heads-up)
        postBlinds();

        // Build combined arrays and sort
        refillCombinedAndSort();

        roundStage = 0;
        statusLabel.setText("Hand dealt. Preflop betting: Small blind acts first.");
        roundLabel.setText("Round: Preflop");
        updateButtons();
        cardPanel.repaint();

        // Start preflop betting with SB acting first
        bettingRoundActive = true;
        int smallBlindSeat = buttonSeat;
        currentActor = smallBlindSeat; // small blind acts first preflop in this simple flow
        SwingUtilities.invokeLater(() -> aiActIfNeeded());
    }

    private void updateButtonLabel() {
        String who = (buttonSeat == humanSeat) ? "You" : "AI";
        buttonLabel.setText("Button: " + who);
    }
```

```
    private void postBlinds() {
        int sbSeat = buttonSeat;
        int bbSeat = 1 - buttonSeat;

        Player sbPlayer = table.get(sbSeat);
        Player bbPlayer = table.get(bbSeat);

        sbPlayer.chipstack -= SMALL_BLIND;
        sbPlayer.contributed = SMALL_BLIND;

        bbPlayer.chipstack -= BIG_BLIND;
        bbPlayer.contributed = BIG_BLIND;

        pot = SMALL_BLIND + BIG_BLIND;
        currentBet = BIG_BLIND;

        // update references
        updateStackLabels();
        updatePotLabel();

        // If someone busted by blinds, handle it (very rare)
        checkForMatchEnd();
    }

    // Advance stage automatically when a betting round finishes
    private void advanceStageIfNeeded() {
        // Called when bettingRoundActive becomes false
        if (roundStage == 0) {
            // preflop -> flop
            roundStage = 1;
            roundLabel.setText("Round: Flop");
            statusLabel.setText("Flop revealed.");
            resetContributionsForNextRound();
            currentActor = buttonSeat; // keep SB acting first postflop in this simplified flow
            bettingRoundActive = true;
            updateButtons();
            cardPanel.repaint();
            SwingUtilities.invokeLater(() -> aiActIfNeeded());
            return;
        } else if (roundStage == 1) {
            // flop -> turn
            roundStage = 2;
            roundLabel.setText("Round: Turn");
            statusLabel.setText("Turn revealed.");
            resetContributionsForNextRound();
            currentActor = buttonSeat;
            bettingRoundActive = true;
            updateButtons();
            cardPanel.repaint();
            SwingUtilities.invokeLater(() -> aiActIfNeeded());
            return;
        } else if (roundStage == 2) {
            // turn -> river
            roundStage = 3;
            roundLabel.setText("Round: River");
            statusLabel.setText("River revealed.");
            resetContributionsForNextRound();
            currentActor = buttonSeat;
            bettingRoundActive = true;
            updateButtons();
            cardPanel.repaint();
            SwingUtilities.invokeLater(() -> aiActIfNeeded());
```

```
            return;
        } else if (roundStage == 3) {
            // river -> showdown
            roundStage = 4;
            roundLabel.setText("Round: Showdown");
            bettingRoundActive = false;
            showdownReveal = true;
            refillCombinedAndSort();
            evaluateHandsAll();
            int winnerIdx = whowonthishand11();
            if (winnerIdx == -1) {
                JOptionPane.showMessageDialog(this, "No active players. Unexpected.");
                javax.swing.Timer t = new javax.swing.Timer(700, e -> dealNewHand());
                t.setRepeats(false);
                t.start();
                return;
            }
            Player winner = table.get(winnerIdx);
            String seat = seatNameOfIndex(winnerIdx);
            JOptionPane.showMessageDialog(this,
                    "Showdown complete.\nWinner: Seat " + winnerIdx + " (" + seat + ")\nHand: " + winner);
            statusLabel.setText("Showdown complete. Winner: " + seat);
            // award pot to winner and immediately prepare next hand
            winner.chipstack += pot;
            pot = 0;
            updateStackLabels();
            updatePotLabel();
            updateButtons();
            cardPanel.repaint();

            // Check for match end
            if (checkForMatchEnd()) return;

            // small delay then auto-deal next hand
            javax.swing.Timer t = new javax.swing.Timer(900, e -> dealNewHand());
            t.setRepeats(false);
            t.start();
        }
    }

    private void resetContributionsForNextRound() {
        for (Player p : table) p.contributed = 0;
        currentBet = 0;
        updateStackLabels();
        updatePotLabel();
        toCallLabel.setText("To Call: $0");
    }

    private void showdown() {
        // not used (auto)
    }

    private String seatNameOfIndex(int idx) {
        switch (idx) {
            case 0: return "AI";
            case 1: return "You";
            default: return "?";
        }
    }

    private void updateButtons() {
        // Deal allowed when match not over (disable after match end)
```

```
        boolean matchOver = (aiPlayer.chipstack <= 0 || humanPlayer.chipstack <= 0);
        dealBtn.setEnabled(!matchOver && (roundStage == 0 || roundStage == 4));

        // Hide action buttons unless it's human's turn and betting is active and human not folded
        boolean humanTurnNow = (currentActor == humanSeat) && bettingRoundActive && !humanPlayer.didifold;
        checkBtn.setEnabled(humanTurnNow && (humanPlayer.contributed == currentBet));
        callBtn.setEnabled(humanTurnNow && (humanPlayer.contributed < currentBet));
        raiseBtn.setEnabled(humanTurnNow && !humanPlayer.didifold);
        foldBtn.setEnabled(humanTurnNow && !humanPlayer.didifold);

        // update HUD call amount
        int toCall = currentBet - humanPlayer.contributed;
        if (toCall < 0) toCall = 0;
        toCallLabel.setText("To Call: $" + toCall);

        // If match over, disable action buttons too
        if (matchOver) {
            checkBtn.setEnabled(false);
            callBtn.setEnabled(false);
            raiseBtn.setEnabled(false);
            foldBtn.setEnabled(false);
            statusLabel.setText("Match over. " + (aiPlayer.chipstack <= 0 ? "You win!" : "AI wins!"));
        }
    }

    // Build combined lists and sort
    private void refillCombinedAndSort() {
        p1.clear(); p2.clear();

        addAll(p1, table.get(0)); addAll(p2, table.get(1));

        Collections.sort(p1); Collections.sort(p2);
    }

    private void addAll(List<Card> dst, Player p) {
        dst.add(comm.card1); dst.add(comm.card2); dst.add(comm.card3);
        dst.add(comm.card4); dst.add(comm.card5);
        dst.add(p.card1); dst.add(p.card2);
    }

    // Hand evaluation
    private void evaluateHandsAll() {
        // Reset straight indices
        idxStr1 = straighthelper(p1, table.get(0));
        idxStr2 = straighthelper(p2, table.get(1));

        // Flush checks
        helperforflush(p1, table.get(0));
        helperforflush(p2, table.get(1));

        // Pairs and multiples
        pairhelper(p1, table.get(0));
        pairhelper(p2, table.get(1));

        twopairhelper(p1, table.get(0));
        twopairhelper(p2, table.get(1));

        threekindhelper(p1, table.get(0));
        threekindhelper(p2, table.get(1));

        fullhousehelper(p1, table.get(0));
        fullhousehelper(p2, table.get(1));
```

```
        fourofakindhelper(p1, table.get(0));
        fourofakindhelper(p2, table.get(1));

        // Straight flush and royal flush
        strflushhelper(p1, table.get(0));
        strflushhelper(p2, table.get(1));

        royalflush(p1, table.get(0));
        royalflush(p2, table.get(1));

        // High card only if no made hand
        highcardhelper(p1, table.get(0));
        highcardhelper(p2, table.get(1));
    }

    private int getHandCategory(Player p) {
        if (p.Royal)      return 9;
        if (p.StrFlush)   return 8;
        if (p.FourKind)   return 7;
        if (p.Fullhouse)  return 6;
        if (p.Flush)      return 5;
        if (p.Straight)   return 4;
        if (p.threekind)  return 3;
        if (p.twopair)    return 2;
        if (p.pair)       return 1;
        if (p.highcard)   return 0;
        return -1;
    }

    private int whowonthishand11() {
        int bestIdx = -1;
        int bestCat = -1;
        for (int i=0;i<table.size();i++) {
            Player p = table.get(i);
            if (p.didifold) continue;
            int cat = getHandCategory(p);
            if (cat > bestCat) {
                bestCat = cat;
                bestIdx = i;
            } else if (cat == bestCat) {
                // tie-breakers not implemented; choose the one with higher top card heuristically
                int bestHigh = bestIdx == -1 ? -1 : highestRankOf(table.get(bestIdx));
                int curHigh = highestRankOf(p);
                if (curHigh > bestHigh) bestIdx = i;
            }
        }
        return bestIdx;
    }

    private int highestRankOf(Player p) {
        int highest = -1;
        if (comm.card1 != null) highest = Math.max(highest, rank(comm.card1));
        if (comm.card2 != null) highest = Math.max(highest, rank(comm.card2));
        if (comm.card3 != null) highest = Math.max(highest, rank(comm.card3));
        if (comm.card4 != null) highest = Math.max(highest, rank(comm.card4));
        if (comm.card5 != null) highest = Math.max(highest, rank(comm.card5));
        if (p.card1 != null) highest = Math.max(highest, rank(p.card1));
        if (p.card2 != null) highest = Math.max(highest, rank(p.card2));
        return highest;
    }
```

```
    // Helpers
    private int highcardhelper(List<Card> cards, Player p) {
        if (p.pair || p.twopair || p.threekind || p.Straight ||
            p.Flush || p.Fullhouse || p.FourKind || p.StrFlush || p.Royal) {
            p.highcard = false;
            return -1;
        }
        int highest = -1;
        for (Card c : cards) highest = Math.max(highest, rank(c));
        p.highcard = true;
        return highest;
    }

    private int pairhelper(List<Card> cards, Player p) {
        p.pair = false;
        int[] cnt = countRanks(cards);
        int pr = -1;
        for (int r=2;r<=14;r++) {
            if (cnt[r]==2) { pr = r; p.pair = true; }
        }
        return pr;
    }

    private int[] twopairhelper(List<Card> cards, Player p) {
        p.twopair = false;
        int[] cnt = countRanks(cards);
        int high=-1, low=-1;
        for (int r=14;r>=2;r--) {
            if (cnt[r]==2) {
                if (high==-1) high=r;
                else if (low==-1) low=r;
            }
        }
        if (high!=-1 && low!=-1) {
            p.twopair = true;
            return new int[]{high,low};
        }
        return new int[]{-1,-1};
    }

    private int threekindhelper(List<Card> cards, Player p) {
        p.threekind = false;
        int[] cnt = countRanks(cards);
        int trips=-1;
        for (int r=14;r>=2;r--) if (cnt[r]==3) { trips=r; break; }
        if (trips==-1) return -1;

        // exclude full house/quads
        boolean hasPairElsewhere = false;
        for (int r=14;r>=2;r--) {
            if (r!=trips && cnt[r]>=2) { hasPairElsewhere=true; break; }
        }
        if (hasPairElsewhere) { p.threekind=false; return -1; }
        p.threekind = true;
        return trips;
    }

    private int[] fullhousehelper(List<Card> cards, Player p) {
        p.Fullhouse = false;
        int[] cnt = countRanks(cards);
        int trips=-1, pair=-1;
        for (int r=14;r>=2;r--) if (cnt[r]==3) { trips=r; break; }
```

```
        if (trips==-1) return new int[]{-1,-1};
        for (int r=14;r>=2;r--) {
            if (r==trips) continue;
            if (cnt[r]>=2) { pair=r; break; }
        }
        if (pair!=-1) { p.Fullhouse=true; return new int[]{trips,pair}; }
        return new int[]{-1,-1};
    }

    private int[] fourofakindhelper(List<Card> cards, Player p) {
        p.FourKind = false;
        int[] cnt = countRanks(cards);
        int quad=-1, kicker=-1;
        for (int r=14;r>=2;r--) if (cnt[r]==4) { quad=r; break; }
        if (quad==-1) return new int[]{-1,-1};
        for (int r=14;r>=2;r--) {
            if (r==quad) continue;
            if (cnt[r]>0) { kicker=r; break; }
        }
        p.FourKind = true;
        return new int[]{quad,kicker};
    }

    private void helperforflush(List<Card> cards, Player p) {
        int h=0,s=0,c=0,d=0;
        for (Card card : cards) {
            switch (card.suit) {
                case "H": h++; break;
                case "S": s++; break;
                case "C": c++; break;
                case "D": d++; break;
            }
        }
        if (h>=5 || s>=5 || c>=5 || d>=5) p.Flush = true;
    }

    private int straighthelper(List<Card> cards, Player p) {
        p.Straight = false;
        if (cards.size()<5) return -1;

        // unique ranks ascending
        List<Integer> uniq = new ArrayList<>();
        int last=-1;
        for (Card card : cards) {
            int r = rank(card);
            if (r!=last) { uniq.add(r); last=r; }
        }
        if (uniq.isEmpty()) return -1;

        // detect run of 5
        int run=1;
        for (int i=1;i<uniq.size();i++) {
            if (uniq.get(i) == uniq.get(i-1)+1) {
                run++;
            } else {
                run=1;
            }
            if (run>=5) { p.Straight=true; return i-4; }
        }
        return -1;
    }
```

```
    private void strflushhelper(List<Card> cards, Player p) {
        p.StrFlush = false;
        if (cards.size()<5) return;

        List<Card> hearts = new ArrayList<>(), spades=new ArrayList<>(),
                   clubs=new ArrayList<>(), diamonds=new ArrayList<>();
        for (Card c : cards) {
            switch (c.suit) {
                case "H": hearts.add(c); break;
                case "S": spades.add(c); break;
                case "C": clubs.add(c); break;
                case "D": diamonds.add(c); break;
            }
        }
        Collections.sort(hearts); Collections.sort(spades);
        Collections.sort(clubs);  Collections.sort(diamonds);

        if (hasStraightInSuit(hearts) || hasStraightInSuit(spades)
                || hasStraightInSuit(clubs) || hasStraightInSuit(diamonds)) {
            p.StrFlush = true;
        }
    }

    private boolean hasStraightInSuit(List<Card> suitCards) {
        if (suitCards.size()<5) return false;
        int run=1;
        for (int i=1;i<suitCards.size();i++) {
            int prev = rank(suitCards.get(i-1));
            int curr = rank(suitCards.get(i));
            if (curr==prev+1) run++;
            else if (curr==prev) continue;
            else run=1;
            if (run>=5) return true;
        }
        return false;
    }

    private void royalflush(List<Card> cards, Player p) {
        // Need 10,J,Q,K,A of same suit present
        Set<String> suits = new HashSet<>();
        for (Card c : cards) if (c.value.equals("10")) suits.add(c.suit);
        for (String suit : suits) {
            boolean hasJ=false,hasQ=false,hasK=false,hasA=false;
            for (Card c : cards) {
                if (!c.suit.equals(suit)) continue;
                if (c.value.equals("J")) hasJ=true;
                else if (c.value.equals("Q")) hasQ=true;
                else if (c.value.equals("K")) hasK=true;
                else if (c.value.equals("A")) hasA=true;
            }
            if (hasJ && hasQ && hasK && hasA) { p.Royal=true; return; }
        }
        p.Royal=false;
    }

    private int[] countRanks(List<Card> cards) {
        int[] cnt = new int[15];
        for (Card c : cards) cnt[rank(c)]++;
        return cnt;
    }

    private int rank(Card c) {
```

```
        switch (c.value) {
            case "2": return 2; case "3": return 3; case "4": return 4; case "5": return 5;
            case "6": return 6; case "7": return 7; case "8": return 8; case "9": return 9;
            case "10": return 10; case "J": return 11; case "Q": return 12;
            case "K": return 13; case "A": return 14;
            default: return -1;
        }
    }

    // ===== Betting logic (which was difficult) =====
■//coming back to this I believe because as of right now the AI will randomly fold a good hand

    private void aiActIfNeeded() {
        if (!bettingRoundActive) return;
        if (currentActor != aiSeat) return; // not AI's turn
        if (table.get(aiSeat).didifold) {
            // AI folded earlier; end betting
            bettingRoundActive = false;
            statusLabel.setText("AI folded earlier.");
            updateButtons();
            // award pot and deal new hand to other
            table.get(humanSeat).chipstack += pot;
            pot = 0;
            updateStackLabels();
            updatePotLabel();
            // check for match end
            if (checkForMatchEnd()) return;
            // auto-deal next
            javax.swing.Timer t = new javax.swing.Timer(500, e -> dealNewHand());
            t.setRepeats(false);
            t.start();
            return;
        }

        // Evaluate AI hand strength (use evaluation routines on combined)
        refillCombinedAndSort();
        evaluateHandsAll();
        int aiCategory = getHandCategory(table.get(aiSeat)); // 0..9

        int toCall = currentBet - table.get(aiSeat).contributed;
        Random rnd = new Random();

        // Basic power-based decision:
        // - Strong made hands (2+ category, i.e., Two Pair or better): raise aggressively.
        // - Medium (pair or high card with high rank): call small, sometimes raise.
        // - Weak: fold to large bets, call small sometimes.

        String actionText = "";
        if (toCall <= 0) {
            // No bet to call: decide check or bet
            if (aiCategory >= 2) {
                // made hand: small raise  (raise between BIG_BLIND and 3*BIG_BLIND)
                int raiseAmt = Math.min((int)table.get(aiSeat).chipstack, BIG_BLIND * (1 + rnd.nextInt(3)));
                doAiRaise(raiseAmt);
                actionText = "AI bets $" + currentBet + ".";
            } else if (aiCategory == 1) {
                // one pair: 40% chance to bet
                if (rnd.nextInt(100) < 40 && table.get(aiSeat).chipstack > BIG_BLIND) {
                    int raiseAmt = Math.min((int)table.get(aiSeat).chipstack, BIG_BLIND);
                    doAiRaise(raiseAmt);
                    actionText = "AI bets $" + currentBet + ".";
                } else {
```

```
                actionText = "AI checks.";
                statusLabel.setText("AI checks.");
                currentActor = 1 - aiSeat;
            }
        } else {
            // weak: mostly check, occasional bluff rarely
            if (rnd.nextInt(100) < 8 && table.get(aiSeat).chipstack > BIG_BLIND*2) {
                int raiseAmt = BIG_BLIND*2;
                doAiRaise(raiseAmt);
                actionText = "AI bluffs and bets $" + currentBet + ".";
            } else {
                actionText = "AI checks.";
                statusLabel.setText("AI checks.");
                currentActor = 1 - aiSeat;
            }
        }
    } else {
        // there's a bet to call
        if (aiCategory >= 3) {
            // trips or better: call and often raise
            if (table.get(aiSeat).chipstack >= toCall) {
                doAiCall();
                // sometimes raise on top
                if (rnd.nextInt(100) < 50 && table.get(aiSeat).chipstack > BIG_BLIND) {
                    int raiseAmt = Math.min((int)table.get(aiSeat).chipstack, BIG_BLIND * (1 + rnd.nextInt(3)));
                    doAiRaise(raiseAmt);
                    actionText = "AI called then raised to $" + currentBet + ".";
                } else {
                    actionText = "AI calls.";
                }
            } else {
                // all-in call
                doAiCall();
                actionText = "AI calls all-in.";
            }
        } else if (aiCategory == 2 || aiCategory == 1) {
            // Two pair or one pair: call small bets, fold to bigs sometimes
            if (toCall <= BIG_BLIND*2 || (rnd.nextInt(100) < 60 && table.get(aiSeat).chipstack >= toCall)) {
                doAiCall();
                actionText = "AI calls $" + Math.min(toCall, (int)table.get(aiSeat).chipstack + toCall) + ".";
            } else {
                table.get(aiSeat).didifold = true;
                actionText = "AI folds to bet of $" + toCall + ".";
                // award pot to human instantly
                table.get(humanSeat).chipstack += pot;
                pot = 0;
                updateStackLabels();
                updatePotLabel();
                bettingRoundActive = false;
                currentActor = -1;
                updateButtons();
                cardPanel.repaint();
                JOptionPane.showMessageDialog(this, actionText);
                if (checkForMatchEnd()) return;
                // auto-deal
                javax.swing.Timer t = new javax.swing.Timer(700, e -> dealNewHand());
                t.setRepeats(false);
                t.start();
                return;
            }
        } else {
            // weak: fold unless small bet
```

```
            if (toCall <= BIG_BLIND && table.get(aiSeat).chipstack >= toCall && rnd.nextInt(100) < 40) {
                doAiCall();
                actionText = "AI calls small $" + Math.min(toCall, (int)table.get(aiSeat).chipstack + toCall) + ".";
            } else {
                table.get(aiSeat).didifold = true;
                actionText = "AI folds to bet of $" + toCall + ".";
                // award pot to human instantly
                table.get(humanSeat).chipstack += pot;
                pot = 0;
                updateStackLabels();
                updatePotLabel();
                bettingRoundActive = false;
                currentActor = -1;
                updateButtons();
                cardPanel.repaint();
                JOptionPane.showMessageDialog(this, actionText);
                if (checkForMatchEnd()) return;
                // auto-deal
                javax.swing.Timer t = new javax.swing.Timer(700, e -> dealNewHand());
                t.setRepeats(false);
                t.start();
                return;
            }
        }
    }

    // Show AI action popup if not already shown due to folding path above
    if (!actionText.isEmpty()) {
        JOptionPane.showMessageDialog(this, actionText);
    }

    // After AI acted, check for all-in showdown
    if (checkAndHandleAllIn()) return;

    // After AI acted, if betting round is still active and it's human's turn, update buttons
    if (bettingRoundActive) {
        currentActor = humanSeat;
        updateButtons();
        cardPanel.repaint();
    } else {
        // bettingRoundActive false -> advance stage automatically
        updateButtons();
        cardPanel.repaint();
        // small delay then advance stage
        javax.swing.Timer t = new javax.swing.Timer(400, e -> advanceStageIfNeeded());
        t.setRepeats(false);
        t.start();
    }
}

private void doAiCall() {
    int toCall = currentBet - table.get(aiSeat).contributed;
    int take = Math.min(toCall, (int)table.get(aiSeat).chipstack);
    table.get(aiSeat).chipstack -= take;
    table.get(aiSeat).contributed += take;
    pot += take;
    statusLabel.setText("AI calls $" + take + ".");
    updateStackLabels();
    updatePotLabel();

    // After AI called, check for all-in showdown
    if (checkAndHandleAllIn()) return;
```

```
        // if both players now have equal contributed amounts then betting round ends
        if (table.get(aiSeat).contributed == table.get(humanSeat).contributed) {
            bettingRoundActive = false;
            // small delay to allow UI to update, then advance stage
            javax.swing.Timer t = new javax.swing.Timer(300, e -> advanceStageIfNeeded());
            t.setRepeats(false);
            t.start();
        }
    }

    private void doAiRaise(int raiseAmount) {
        // raiseAmount is the additional amount AI wants to increase currentBet by
        int newBet = currentBet + raiseAmount;
        int toCall = newBet - table.get(aiSeat).contributed;
        int take = Math.min(toCall, (int)table.get(aiSeat).chipstack);
        table.get(aiSeat).chipstack -= take;
        table.get(aiSeat).contributed += take;
        pot += take;
        currentBet = Math.max(currentBet, table.get(aiSeat).contributed);
        statusLabel.setText("AI raises to $" + currentBet + ".");
        updateStackLabels();
        updatePotLabel();

        // after raise, betting remains active and human must respond
        bettingRoundActive = true;
        currentActor = humanSeat;
        updateButtons();
        cardPanel.repaint();

        // After raise, check all-in (if AI shoved)
        checkAndHandleAllIn();
    }

    // Human action handlers
    private void humanCheck() {
        if (currentActor != humanSeat || !bettingRoundActive) return;
        if (table.get(humanSeat).contributed != currentBet) {
            statusLabel.setText("Cannot check — there is a bet to call.");
            return;
        }
        statusLabel.setText("You check.");
        // If both have equal contributed amounts, betting round ends
        if (table.get(humanSeat).contributed == table.get(aiSeat).contributed) {
            bettingRoundActive = false;
            updateButtons();
            cardPanel.repaint();
            javax.swing.Timer t = new javax.swing.Timer(300, e -> advanceStageIfNeeded());
            t.setRepeats(false);
            t.start();
        } else {
            // else give turn to AI
            currentActor = aiSeat;
            updateButtons();
            cardPanel.repaint();
            SwingUtilities.invokeLater(() -> aiActIfNeeded());
        }
    }

    private void humanCall() {
        if (currentActor != humanSeat || !bettingRoundActive) return;
        int toCall = currentBet - table.get(humanSeat).contributed;
```

```
        int take = Math.min(toCall, (int)table.get(humanSeat).chipstack);
        table.get(humanSeat).chipstack -= take;
        table.get(humanSeat).contributed += take;
        pot += take;
        statusLabel.setText("You call $" + take + ".");
        updateStackLabels();
        updatePotLabel();

        // After human called, check for all-in showdown
        if (checkAndHandleAllIn()) return;

        // if both contributions equal -> betting round ends and advance stage
        if (table.get(humanSeat).contributed == table.get(aiSeat).contributed) {
            bettingRoundActive = false;
            updateButtons();
            cardPanel.repaint();
            javax.swing.Timer t = new javax.swing.Timer(300, e -> advanceStageIfNeeded());
            t.setRepeats(false);
            t.start();
        } else {
            // otherwise it's AI turn
            currentActor = aiSeat;
            updateButtons();
            SwingUtilities.invokeLater(() -> aiActIfNeeded());
        }
    }

    private void humanRaise() {
        if (currentActor != humanSeat || !bettingRoundActive) return;
        int raiseAmt;
        try {
            raiseAmt = Integer.parseInt(raiseField.getText().trim());
            if (raiseAmt <= 0) { statusLabel.setText("Raise must be > 0."); return; }
        } catch (NumberFormatException ex) {
            statusLabel.setText("Invalid raise amount.");
            return;
        }
        // No-limit: newBet = currentBet + raiseAmt
        int newBet = currentBet + raiseAmt;
        int toCall = newBet - table.get(humanSeat).contributed;
        if (toCall <= 0) toCall = raiseAmt; // fallback
        int take = Math.min(toCall, (int)table.get(humanSeat).chipstack);
        table.get(humanSeat).chipstack -= take;
        table.get(humanSeat).contributed += take;
        pot += take;
        currentBet = Math.max(currentBet, table.get(humanSeat).contributed);

        statusLabel.setText("You raise to $" + currentBet + ".");
        updateStackLabels();
        updatePotLabel();

        // back to AI to respond
        currentActor = aiSeat;
        updateButtons();
        cardPanel.repaint();
        // show human raise popup then AI acts
        JOptionPane.showMessageDialog(this, "You raised to $" + currentBet + ".");

        // After human raised, check for all-in showdown (if human shoved)
        if (checkAndHandleAllIn()) return;

        SwingUtilities.invokeLater(() -> aiActIfNeeded());
```

```
    }

    private void humanFold() {
        if (currentActor != humanSeat || !bettingRoundActive) return;
        table.get(humanSeat).didifold = true;
        statusLabel.setText("You fold. AI wins the pot.");
        // award pot to AI immediately
        table.get(aiSeat).chipstack += pot;
        pot = 0;
        updateStackLabels();
        updatePotLabel();
        bettingRoundActive = false;
        updateButtons();
        cardPanel.repaint();
        JOptionPane.showMessageDialog(this, "You folded. AI wins the hand.");
        // Check match end
        if (checkForMatchEnd()) return;
        // auto-deal new hand shortly
        javax.swing.Timer t = new javax.swing.Timer(700, e -> dealNewHand());
        t.setRepeats(false);
        t.start();
    }

    private void updateStackLabels() {
        aiStackLabel.setText("AI: $" + (int)table.get(aiSeat).chipstack);
        humanStackLabel.setText("You: $" + (int)table.get(humanSeat).chipstack);
    }

    private void updatePotLabel() {
        potLabel.setText("Pot: $" + pot);
    }

    // returns true if match ended (and UI updated)
    private boolean checkForMatchEnd() {
        if (table.get(aiSeat).chipstack <= 0) {
            // Human wins match
            JOptionPane.showMessageDialog(this, "You have won the match! AI is out of chips.");
            disableGameAfterMatch();
            return true;
        }
        if (table.get(humanSeat).chipstack <= 0) {
            JOptionPane.showMessageDialog(this, "AI has won the match! You are out of chips.");
            disableGameAfterMatch();
            return true;
        }
        return false;
    }

    private void disableGameAfterMatch() {
        // Disable buttons and prevent new deals so you can't play with 0 money
        dealBtn.setEnabled(false);
        checkBtn.setEnabled(false);
        callBtn.setEnabled(false);
        raiseBtn.setEnabled(false);
        foldBtn.setEnabled(false);
        statusLabel.setText("Match over.");
    }

    /**
     * check if all-in was reached and if both sides have matched contributions.
     * If so, force a showdown immediately (reveal remaining cards and evaluate).
     * Returns true if an all-in showdown
```

```
     */
    private boolean checkAndHandleAllIn() {
        // a player is all-in if chipstack == 0
        boolean humanAllIn = table.get(humanSeat).chipstack <= 0;
        boolean aiAllIn = table.get(aiSeat).chipstack <= 0;

        // If neither is all-in, nothing to do here.
        if (!humanAllIn && !aiAllIn) return false;

        // If both have equal contributed amounts (i.e., all-in was called or both all-in), trigger showdown.
        if (table.get(humanSeat).contributed == table.get(aiSeat).contributed) {
            // disable betting controls immediately
            bettingRoundActive = false;
            checkBtn.setEnabled(false);
            callBtn.setEnabled(false);
            raiseBtn.setEnabled(false);
            foldBtn.setEnabled(false);

            // handle forced showdown
            handleAllInShowdown();
            return true;
        }

        // If one side is all-in but the other hasn't called yet, wait for call/fold.
        return false;
    }

    /**
     * Force-reveal remaining streets and do showdown immediately.
     * This is called when all-in has been reached and the all-in has been called.
     */
    private void handleAllInShowdown() {
        // Reveal remaining cards immediately
        roundStage = 3; // ensure river will be visible
        roundLabel.setText("Round: Showdown (all-in)");
        showdownReveal = true;

        // Evaluate hands & determine winner
        refillCombinedAndSort();
        evaluateHandsAll();
        int winnerIdx = whowonthishand11();
        if (winnerIdx == -1) {
            JOptionPane.showMessageDialog(this, "No active players. Unexpected.");
            javax.swing.Timer t = new javax.swing.Timer(700, e -> dealNewHand());
            t.setRepeats(false);
            t.start();
            return;
        }
        Player winner = table.get(winnerIdx);
        String seat = seatNameOfIndex(winnerIdx);

        // Showpopup with both players' actions and the showdown result
        JOptionPane.showMessageDialog(this, "All-in showdown!\nWinner: " + seat + "\nHand: " + winner);

        // Award pot
        winner.chipstack += pot;
        pot = 0;
        updateStackLabels();
        updatePotLabel();
        updateButtons();
        cardPanel.repaint();
```

```
        // Check for match end
        if (checkForMatchEnd()) return;

        // small delay then auto-deal next hand
        javax.swing.Timer t = new javax.swing.Timer(900, e -> dealNewHand());
        t.setRepeats(false);
        t.start();
    }


    class CardPanel extends JPanel {
        @Override
        protected void paintComponent(Graphics g) {
            super.paintComponent(g);
            setBackground(new Color(0, 128, 0)); // green table

            // Draw AI at top middle
            drawPlayerTop(g, table.get(0), "AI", 370, 40);

            // Draw community in center
            int cx = 250, cy = 260;
            if (roundStage >= 1) drawCard(g, comm.card1, cx, cy);
            if (roundStage >= 1) drawCard(g, comm.card2, cx+90, cy);
            if (roundStage >= 1) drawCard(g, comm.card3, cx+180, cy);
            if (roundStage >= 2) drawCard(g, comm.card4, cx+270, cy);
            if (roundStage >= 3) drawCard(g, comm.card5, cx+360, cy);

            // Draw human at bottom middle
            drawPlayerBottom(g, table.get(1), "YOU", 370, 520);
        }

        private void drawPlayerTop(Graphics g, Player p, String label, int x, int y) {
            g.setColor(Color.WHITE);
            g.setFont(new Font("SansSerif", Font.BOLD, 16));
            g.drawString(label, x, y-10);

            // if showdownReveal true OR opponent folded reveal; else hide
            if (showdownReveal || p.didifold) {
                drawCard(g, p.card1, x, y);
                drawCard(g, p.card2, x+90, y);
            } else {
                // draw card backs
                drawCardBack(g, x, y);
                drawCardBack(g, x+90, y);
            }

            g.setColor(Color.WHITE);
            String cat = handCategoryName(p);
            g.drawString(cat, x, y+120);

            // show chip stack and contributed
            g.drawString("Stack: $" + (int)p.chipstack, x, y+140);
            g.drawString("Contrib: $" + p.contributed, x, y+160);
        }

        private void drawPlayerBottom(Graphics g, Player p, String label, int x, int y) {
            g.setColor(Color.WHITE);
            g.setFont(new Font("SansSerif", Font.BOLD, 16));
            g.drawString(label, x, y-10);

            // Human cards visible always
            drawCard(g, p.card1, x, y);
```

```
            drawCard(g, p.card2, x+90, y);

            g.setColor(Color.WHITE);
            String cat = handCategoryName(p);
            g.drawString(cat, x, y+120);

            // show chip stack and contributed
            g.drawString("Stack: $" + (int)p.chipstack, x, y+140);
            g.drawString("Contrib: $" + p.contributed, x, y+160);
        }

        private void drawCardBack(Graphics g, int x, int y) {
            g.setColor(Color.LIGHT_GRAY);
            g.fillRoundRect(x, y, 70, 100, 12, 12);
            g.setColor(Color.BLACK);
            g.drawRoundRect(x, y, 70, 100, 12, 12);
            g.setFont(new Font("SansSerif", Font.BOLD, 18));
            g.drawString("??", x+20, y+55);
        }

        private void drawCard(Graphics g, Card card, int x, int y) {
            if (card == null) return;
            g.setColor(Color.WHITE);
            g.fillRoundRect(x, y, 70, 100, 12, 12);
            g.setColor(Color.BLACK);
            g.drawRoundRect(x, y, 70, 100, 12, 12);

            // convert suit letter to symbol color
            char sym = suitSymbol(card.suit);
            Color suitColor = (sym=='♥' || sym=='♦') ? Color.RED : Color.BLACK;

            g.setColor(suitColor);
            g.setFont(new Font("SansSerif", Font.BOLD, 18));
            g.drawString(card.value, x+8, y+40);
            g.drawString(String.valueOf(sym), x+8, y+70);
        }

        private char suitSymbol(String suitLetter) {
            switch (suitLetter) {
                case "H": return '♥';
                case "D": return '♦';■■//unicode was available for suits which is pretty neat
                case "S": return '♠';
                case "C": return '♣';
                default: return '?';
            }
        }

        private String handCategoryName(Player p) {
            if (p.Royal) return "Royal Flush";
            if (p.StrFlush) return "Straight Flush";
            if (p.FourKind) return "Four of a Kind";
            if (p.Fullhouse) return "Full House";
            if (p.Flush) return "Flush";
            if (p.Straight) return "Straight";
            if (p.threekind) return "Three of a Kind";
            if (p.twopair) return "Two Pair";
            if (p.pair) return "Pair";
            if (p.highcard) return "High Card";
            return "";
        }
    }
```

```
    public static void main(String[] args) {
        SwingUtilities.invokeLater(() -> {
            PokerGUI gui = new PokerGUI();
            gui.setVisible(true);
        });
    }
}
```